Table of Contents

VSYNC and Fences in Android System

A comprehensive deep dive into display synchronization, timing, and buffer lifecycle in the Android graphics pipeline.

Table of Contents

Prerequisites

Display Refresh Cycle

Modern displays update pixels row-by-row from top to bottom. A **refresh cycle** (or frame) is one full screen update:

Refresh rate	Frames per second (e.g., 60 Hz = 60 frames/sec)
Frame period	Time per frame (e.g., 60 Hz → ~16.67 ms)
Vertical blanking	Brief interval between frames when no pixels are being drawn

```
Frame N Frame N+1 |-------------------------|-------------------------| |<-- ~16.67 ms @ 60 Hz
--->| ^ VSYNC pulse (sync point)
```

Producer-Consumer Model

Android uses a producer-consumer pattern for graphics:

- **Producer**: App (GPU) renders into a buffer
- **Consumer**: Display subsystem (SurfaceFlinger, HWC) reads buffer and shows it
- **BufferQueue**: Mediates between producer and consumer

Triple Buffering (Simplified)

```
[Buffer A: Display] [Buffer B: Queued] [Buffer C: Rendering] Display Waiting App draws
```

- Typically 2–3 buffers in flight
- Allows app to work ahead of the display
- Reduces stalls when render time varies

---

1. VSYNC

What Is VSYNC?

VSYNC (Vertical Synchronization) is a hardware signal that marks the start of each display refresh cycle. It acts as a timing heartbeat for the graphics pipeline.

Source	Display controller (HWC/DRM)
Frequency	Matches display refresh (e.g., 60 Hz, 90 Hz, 120 Hz)
Purpose	Coordinate when apps and SurfaceFlinger start work

HSYNC vs VSYNC

Displays update **row-by-row** (scanlines). There are two fundamental sync signals:

HSYNC (Horizontal Sync)	End of one **scanline** (one horizontal row)	One per row per frame
VSYNC (Vertical Sync)	End of one **frame** (all scanlines)	One per frame

```
VSYNC | v +-----------------------------------------+ | Row 0 <- HSYNC between each row | | Row 1 <-
HSYNC | | Row 2 <- HSYNC | | ... | | Row N <- HSYNC | +-----------------------------------------+ ^
VSYNC (start of next frame)
```

Example (1080p @ 60 Hz):

- **HSYNC**: 1080 × 60 ≈ 64,800 per second
- **VSYNC**: 60 per second

Why VSYNC Is More Prominent Than HSYNC

Relevance to apps	Low	High
Granularity	Per scanline	Per frame
Used for	Display hardware, timing specs	GPU/OS/app scheduling
Tearing	Occurs mid-frame (between rows)	Prevented by syncing at frame boundary

- **Frame-level scheduling**: Apps and GPUs work in full frames, not per-scanline. The natural sync boundary is the frame (VSYNC).
- **Tearing fix**: Tearing happens when the display switches buffers mid-frame. Aligning buffer swaps to VSYNC (frame boundary) fixes it.
- **API design**: Choreographer, present-at-vsync, and display APIs all operate at frame granularity.
- **Where HSYNC matters**: Display controller design, VGA/HDMI timing validation, CRT signals, video capture. Software rarely exposes HSYNC to apps.

Why VSYNC Matters

Without VSYNC:

- Apps render at arbitrary times
- Display may switch buffers mid-frame → **screen tearing**
- Unpredictable latency and jank

With VSYNC:

- Everyone aligns to the same timing boundary
- No tearing; smooth, consistent frame delivery

Software VSYNC vs Hardware VSYNC

Hardware VSYNC	Display controller	Ground truth; one per physical refresh
Software VSYNC (App)	DispSync simulation	Wakes apps for the next frame

Software VSYNC (SF)	DispSync simulation	Wakes SurfaceFlinger for composition

Android often uses **phase offsets** so App and SurfaceFlinger get VSYNC at different points in the pipeline, trading latency for jitter.

Phase Offsets

```
Hardware VSYNC | v |---- vsync_event_phase_offset_ns ---->| App Choreographer wakes |---- vsync_sf_event_phase_offset_ns ->| SurfaceFlinger wakes | [Next frame boundary]
```

`vsync_event_phase_offset_ns`	When Choreographer callback fires for apps
`vsync_sf_event_phase_offset_ns`	When SurfaceFlinger wakes for composition

Default values favor **low jitter** over low latency (~2 frames app-to-display).

Choreographer: App-Side VSYNC

The **Choreographer** is the app-facing VSYNC coordinator. It receives timing pulses and schedules callbacks for the next frame.

```
// Schedule work for the next VSYNC Choreographer.getInstance().postFrameCallback(new Choreographer.FrameCallback() { @Override public void doFrame(long frameTimeNanos) { // Runs on next VSYNC - do rendering here view.invalidate(); // Schedule next frame Choreographer.getInstance().postFrameCallback(this); } });
```

Real-world: Custom render loop

```
public class GameView extends SurfaceView implements Runnable { private Thread renderThread; private volatile boolean running; @Override public void run() { Choreographer choreographer = Choreographer.getInstance(); final Choreographer.FrameCallback callback = new Choreographer.FrameCallback() { @Override public void doFrame(long frameTimeNanos) { if (!running) return; updateGameState(frameTimeNanos); drawFrame(); choreographer.postFrameCallback(this); } }; choreographer.postFrameCallback(callback); } private void drawFrame() { Canvas c = getHolder().lockCanvas(); if (c != null) { // Draw to canvas getHolder().unlockCanvasAndPost(c); } } }
```

postFrameCallback vs postVsyncCallback (API 33+)

`postFrameCallback`	Later in frame	Animations, typical draw sync
`postVsyncCallback`	Start of VSYNC	Early prep, access to `FrameData`

```
// API 33+ - Earlier notification with frame metadata
Choreographer.getInstance().postVsyncCallback(new Choreographer.VsyncCallback() { @Override public void onVsync(long frameTimeNanos, long frameDeadlineNanos, long frameIntervalNanos) { // Can use frameDeadlineNanos for scheduling } });
```

SurfaceFlinger and VSYNC

SurfaceFlinger composites layers on VSYNC boundaries:

1. VSYNC (SF) fires
2. SurfaceFlinger wakes
3. Acquires buffers from BufferQueues (waits on fences if needed)
4. Composites via GL or HWC
5. Presents to display
6. Waits for next VSYNC

---

2. Fences

What Are Fences?

Fences are synchronization primitives that signal when a hardware unit (GPU, display, etc.) has finished work on a buffer. They enable **asynchronous** producer-consumer workflows without busy-waiting.

Unsignaled	Work in progress
Signaled	Work complete
Error	Work failed

Two Main Fence Types

Acquire fence (in-fence)	Consumer waits	Consumer must wait before reading buffer
Release fence (out-fence)	Producer waits	Producer must wait before reusing buffer

```
Producer (App) Consumer (Display) | | |---- queue buffer + release fence ---->| | | | (buffer + fence) | wait on acquire fence | | then read buffer | <---- release fence --------------| (done displaying) | wait before reusing |
```

- **Acquire fence**: "Don't read until GPU is done drawing."
- **Release fence**: "Don't draw again until display is done showing."

Kernel Components

sync_timeline	Monotonically increasing counter per driver (GL, display, etc.)
sync_pt	Fence representing a point on a timeline
sync_file	File descriptor wrapping fence for userspace
dma_fence	Kernel primitive for DMA/GPU sync

sync_file is the carrier between kernel and userspace. Fences are passed as FDs.

Presentation vs Release Fences

Presentation fence	Consumer creates	Signals when buffer is on screen
Release fence	Consumer provides to producer	Signals when consumer is done with buffer

SyncFence API (Android 14+)

```
// SyncFence - Java API for fence handling // Typically used by system components, not app developers directly // HardwareBufferRenderer.RenderResult includes SyncFence for presentation timing
```

App developers usually interact with fences indirectly through Surface, Canvas, and the rendering APIs.

Fence Flow in BufferQueue

```
App (Producer) SurfaceFlinger (Consumer) | | | dequeueBuffer() | | ------------------------------------>| (buffer available) | | | [GPU renders] | | [GPU signals fence when done] | | | | queueBuffer(buffer, releaseFence) | | ------------------------------------>| wait(releaseFence) | | then composite | <--------- acquireFence --------------| (consumer done) | wait(acquireFence) before reusing |
```

Native / HAL Example (Conceptual)

```
// Producer side - after GPU work, attach fence to buffer sp<Fence> releaseFence = getGpuCompletionFence(); // From GPU driver result = producer->queueBuffer(slot, input, releaseFence); // Consumer side - before using buffer, wait on acquire fence sp<Fence> acquireFence = consumer->getCurrentFence(); acquireFence->waitForever(); // Block until buffer is ready
```

Display Pipeline Architecture

End-to-End Flow

```
[App] ---> BufferQueue ---> [SurfaceFlinger] ---> [HWComposer] ---> [Display] | | | | | | | VSYNC Fences VSYNC VSYNC (Choreographer) (SF phase) (hardware)
```

Components

Choreographer	Schedules app work on VSYNC
BufferQueue	Buffers + fences between app and SF
SurfaceFlinger	Composites layers, manages HWC
HWComposer (HWC)	Puts final image on display

SurfaceFlinger Composition

1. VSYNC-SF fires
2. SurfaceFlinger wakes
3. For each visible layer: wait on acquire fence, get buffer

4. Call HWC `prepare()` – HWC decides overlay vs client composition
5. For client layers: GL compose into framebuffer
6. Call HWC `set()` – display composed result
7. Provide release fences to producers

Real-World Use Cases

1. Smooth Animation (Choreographer)

```
ObjectAnimator animator = ObjectAnimator.ofFloat(view, "translationX", 0, 100);
animator.setInterpolator(new LinearInterpolator()); animator.setDuration(300); animator.start(); //
Animation framework uses Choreographer internally for frame-synced updates
```

2. Game Loop with VSYNC

```
// Match display refresh rate DisplayManager dm = (DisplayManager) getSystemService(DISPLAY_SERVICE);
Display display = dm.getDisplay(Display.DEFAULT_DISPLAY); float refreshRate =
display.getRefreshRate(); // e.g., 60 or 90
Choreographer.getInstance().postFrameCallback(frameCallback);
```

3. Custom View Drawing

```
@Override protected void onDraw(Canvas canvas) { super.onDraw(canvas); // This is typically triggered
by View.invalidate() which eventually // schedules a draw on the next VSYNC via Choreographer }
```

4. Systrace / Perfetto Debugging

```
# Capture VSYNC and fence info adb shell perfetto -c config.pb -o trace.pb # In Perfetto UI: enable
"Frame Timeline" and "VSYNC" tracks # Look for: vsync-app, vsync-sf, frame boundaries, fence wait
times
```

Pitfalls and Benefits

Pitfalls

Ignoring VSYNC	Tearing, jank	Use Choreographer or display-linked callbacks
Heavy work on UI thread	Miss VSYNC, dropped frames	Offload to background; keep UI thread < 16 ms

Blocking on fence	Pipeline stalls	Ensure GPU/display complete in time
Wrong phase offsets	Latency or jitter	Use system defaults; tune only with profiling
Triple buffer saturation	Revert to single-buffer, more drops	Optimize render path, reduce overdraw
Implicit sync (legacy)	Desktop-style freezes, device variance	Prefer explicit fencing (modern Android)

Benefits

No tearing	All updates aligned to VSYNC
Predictable timing	Same cadence for apps and composition
Async buffer handoff	Fences allow overlap of render and display
Better debugging	Explicit fences visible in tracing
Cross-device consistency	Explicit sync reduces driver differences

---

Corner Cases and Failure Modes

1. Jank and Dropped Frames

Cause: App or SurfaceFlinger misses the frame deadline.

- Heavy main-thread work
- GPU overload
- Lock contention

Detection: FrameTimeline (Android 12+), Perfetto, "Janky frames" metric.

```
Expected: [====Frame N====] Actual: [====Frame N========] (late) ^ Missed deadline -> jank
```

2. Fence Timeout / Deadlock

Cause: Fence never signals (GPU hang, driver bug).

Impact: App or SF blocks forever; ANR or frozen UI.

Mitigation: Kernel/driver timeouts, watchdog, kill hung process.

3. Phase Offset Misconfiguration

Cause: OEM tweaks phase offsets without validation.

Impact: Increased latency or jitter; inconsistent behavior across devices.

Guidance: Don't change without systrace/automated testing.

4. BufferQueue Exhaustion

Cause: Producer queues faster than consumer; all buffers in use.

Impact: `dequeueBuffer` blocks; potential deadlock if not careful.

Typical: Queue size ~2–3; producer waits for release fence.

5. High Refresh Rate (90/120 Hz)

Cause: Shorter frame period (e.g., 11.1 ms at 90 Hz).

Impact: Easier to miss deadline; need proportionally faster rendering.

6. Multi-Display (e.g., external monitor)

Cause: Different displays can have different VSYNC rates.

Impact: Need per-display VSYNC and composition scheduling.

7. Choreographer Callback Spam

Cause: Posting many callbacks per frame or recursive posts.

Impact: Extra work, possible backlog, missed frames.

```
// BAD: Recursive post without completion guard void doFrame(long t) { heavyWork();
Choreographer.getInstance().postFrameCallback(this); // Posts again immediately }
```

8. Fence FD Leak

Cause: Not closing fence FDs after use.

Impact: FD exhaustion, process or system instability.

Interview Questions & Answers

Q1: What is VSYNC, and why does Android use it?

A: VSYNC is a vertical synchronization signal from the display that marks the start of each refresh cycle. Android uses it so apps and SurfaceFlinger start their work on aligned boundaries, which eliminates screen tearing and reduces jank by keeping the pipeline timed to the display.

Q2: What are acquire and release fences?

A: An **acquire fence** is an in-fence: the consumer must wait on it before reading a buffer (e.g., wait for GPU to finish rendering). A **release fence** is an out-fence: the producer must wait on it before reusing a buffer (e.g., wait for display to finish showing it). Together they coordinate buffer access between producer and consumer.

Q3: What is the Choreographer, and when do you use it?

A: The Choreographer coordinates app work with display refresh. It receives VSYNC (or simulated VSYNC) and runs callbacks on the next frame. Use `postFrameCallback` for animations and custom render loops that need to sync with the display.

Q4: Why does Android use software VSYNC with phase offsets?

A: There is one hardware VSYNC per display refresh. Apps and SurfaceFlinger need to run at different times in the pipeline. Phase offsets create separate software VSYNC events so the app can start rendering earlier and SurfaceFlinger can run later, balancing latency and jitter.

Q5: What is triple buffering, and when can it "fall back"?

A: Triple buffering keeps multiple buffers in flight: one on display, one queued, one being rendered. If the producer is too slow and misses deadlines, the system can effectively reduce to single-buffer mode until it catches up, which may cause more dropped frames.

Q6: How do fences improve performance over implicit synchronization?

A: Implicit sync often uses global flushes that serialize everything. Explicit fences let each buffer carry its own completion signal. Producer and consumer can overlap: e.g., the app can enqueue a buffer with a fence and start the next frame while the display is still finishing the previous one.

Q7: How would you debug a janky frame in Android?

A: Use Perfetto/systrace with FrameTimeline and VSYNC. Check whether the app or SurfaceFlinger missed the deadline, how long fence waits were, and where time was spent (main thread, GPU, etc.). Look for blocking work, lock contention, and overdraw.

Q8: What is the difference between postFrameCallback and postVsyncCallback?

A: `postFrameCallback` runs later in the frame and is the usual choice for animations. `postVsyncCallback` (API 33+) runs at the start of the VSYNC and provides `FrameData`; use it when you need earlier timing or frame metadata.

Q9: What is HSYNC, and why is it less prominent than VSYNC?

A: HSYNC (Horizontal Sync) marks the end of each scanline (row); VSYNC marks the end of each full frame. HSYNC fires thousands of times per second (e.g., 1080 × 60 at 1080p60), while VSYNC fires once per frame. Apps and GPUs schedule at frame granularity, not per-scanline, so the frame boundary (VSYNC) is the

relevant sync point. HSYNC matters for display hardware, timing specs, and video capture, but software APIs focus on VSYNC.
